

Rapport de Projet

Système de Gestion de Fraude (Java – POO)

1. Avancée du projet

Fonctionnalités développées

Le projet implémente un système complet de gestion des fraudes aux examens en Java, structuré selon les principes de la programmation orientée objet.

Architecture du modèle :

- Classe abstraite Fraude (base commune) avec date de relevé, description et contenu
- Quatre sous-classes concrètes : lag (nom du service IA), lagconnecte (adresse IP avec validation IPv4 par regex), Papier (dimension + état plié), Calculatrice (marque)
- Classe Etudiant avec génération automatique d'ID unique (année + compteur)
- Classe Epreuve avec modalité (enum), durée, QCUE et horodatage automatique
- Classe Formulaire reliant étudiants, fraudes et épreuve, avec ID auto-incrémenté
- Deux enums : Enumcursus (B1 à E5a) et Enummodalite (ECRIT, ORAL, QCM, PROJET, TP)

Fonctionnalités de l'application (menu console à 12 options) :

- Recherche d'étudiants par nom, prénom ou ID
- Recherche de formulaires par étudiant ou par épreuve (QCUE)
- Ajout et suppression d'étudiants, formulaires, épreuves et fraudes
- Analyse statistique : nombre total de fraudes, étudiants distincts, moyenne et écart-type
- Génération d'un graphe non orienté des étudiants ayant co-fraudé

Bugs rencontrés et difficultés

- Gestion du buffer du Scanner : des appels successifs à nextInt() puis nextLine() provoquaient des sauts de saisie. Résolu par un appel systématique à scanner.nextLine() dans les blocs finally.
- Un bug a été identifié dans ajouterFormulaire() : les fraudes de type lagConnecte sont ajoutées dans l'ensemble global fraudes au lieu de fraudesForm, les excluant du formulaire créé. Détecté mais non corrigé avant rendu.
- La suppression d'une fraude par description est sensible à la casse et ne gère pas les doublons de description, ce qui peut entraîner une suppression imprévue.
- L'absence de toString() sur Etudiant et Formulaire rend l'affichage peu lisible dans certaines méthodes.

2. Ressources externes et utilisation de l'IAG

Ressources externes utilisées

Les ressources suivantes ont été consultées pour mener à bien le projet :

- Documentation officielle Java (docs.oracle.com) : classes LocalDate, LocalDateTime, ArrayList, HashSet, HashMap, Scanner, expressions régulières
- Cours et supports fournis par l'ESEO : cahier des charges du projet, cours de POO (héritage, classes abstraites, polymorphisme, enums)
- Stack Overflow : résolution de problèmes spécifiques liés au Scanner (gestion du buffer avec nextInt/nextLine) et à la validation d'adresses IP par regex

Utilisation de l'Intelligence Artificielle Générative

L'IA a été utilisée de manière ponctuelle et ciblée, dans une démarche de compréhension et non de délégation. Concrètement :

- Compréhension de concepts : reformulation de notions vues en cours (polymorphisme, classes abstraites), sans génération de code directe
- Débogage assisté : soumission d'extraits de code au chatbot pour obtenir des pistes d'explication sur des comportements inattendus du Scanner
- Aucune fonction du projet n'a été générée intégralement par IA. Tout le code source a été écrit, compris et adapté manuellement par les membres du groupe

Cette utilisation est transparente et mesurée. Elle respecte les consignes du projet tout en s'inscrivant dans une logique d'apprentissage actif.

3. Bilan individuel des contributions

Membre 1 : BODJRENOU

Contribution principale : modélisation du domaine (classes Fraude, Iag, Iagconnecte, Papier, Calculatrice, Epreuve), mise en place de l'architecture d'héritage, rédaction des JavaDocs.

- Conception et implémentation de la hiérarchie de classes (classe abstraite Fraude et ses sous-classes)
- Rédaction des commentaires Javadoc sur l'ensemble des classes du modèle
- Réalisation du diagrammes de classe du modèle
- Réalisation du cahier de test du projet

Membre 2 :DEGUENON Fidel

Contribution principale : développement de l'application principale (SystemeFraude.java), gestion des menus, robustesse des saisies et fonctionnalités avancées.

- Implémentation des méthodes CRUD (ajout, recherche, suppression) pour les 4 entités
- Développement de l'analyse statistique (moyenne, écart-type) et du graphe non orienté
- Mise en place de la validation IPv4 par regex pour les fraudes IagConnecte
- Gestion de la robustesse des saisies (try/catch/finally sur toutes les entrées utilisateur)
- Tests unitaires sur les classes et réalisation du diagrammes de séquences

Note : Ce bilan reflète une répartition équilibrée du travail. La collaboration entre membres a été constante, notamment lors des phases de débogage et de test de l'application.